

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KÌ

Phân tích và tái hiện bài báo SCRFD
Sample and Computation Redistribution for Efficient Face
Detection

Môn học: Nhận dạng

Lớp: 23KHMT

Giảng viên hướng dẫn: Thầy Võ Thanh Lâm
Thầy Lê Hoàng Thái
Thầy Nguyễn Thanh Tình

Sinh viên: Trương Quốc Cường – 23127333
Lê Anh Duy – 23127011
Trần Gia Huy – 23127199

TP. Hồ Chí Minh, ngày 18 tháng 4 năm 2026

Lời mở đầu

Báo cáo cuối kỳ này tập trung vào bài báo *Sample and Computation Redistribution for Efficient Face Detection* (SCRFD) của Guo và cộng sự, công bố tại ICLR 2022. Mục tiêu của báo cáo không chỉ là tóm tắt nội dung paper, mà là trình bày bài báo theo cấu trúc của một báo cáo khoa học hoàn chỉnh: nêu động lực nghiên cứu, phân tích phương pháp, đối chiếu paper với mã nguồn chính thức, tái hiện thực nghiệm và thảo luận ý nghĩa của kết quả thu được.

Từ báo cáo giữa kỳ, nhóm chỉ kế thừa phần liên quan trực tiếp đến Chương II về SCRFD: kiến trúc Backbone–Neck–Head, hai cơ chế *Computation Redistribution* và *Sample Redistribution*, cùng các nhận định về hiệu quả trên WIDER FACE. Phạm vi cuối kỳ được thu hẹp về một bài báo khoa học cụ thể và phần tái hiện thực nghiệm của bài báo đó.

Phần tái hiện chính sử dụng mã nguồn *SCRFD* trong cây `insightface/detection/scrfd`, cùng các script thí nghiệm ở nhánh `experiments` để tổ chức lại các run baseline, paper-SR12 và phần mở rộng *ASR+JSAR*. Nhánh SPOS-NAS proxy được giữ lại như một phân tích phụ để minh họa thêm trực giác về tái phân bổ tính toán, nhưng không thay thế cho phần tái hiện chính bằng official code.

Bố cục báo cáo gồm bảy chương. Chương I giới thiệu bài báo SCRFD và bài toán nghiên cứu. Chương II trình bày phương pháp trong paper. Chương III mô tả quá trình tái hiện mã nguồn chính thức. Chương IV đối chiếu các ý tưởng trong paper với module code tương ứng. Chương V trình bày kết quả thực nghiệm và so sánh với kết quả công bố. Chương VI thảo luận các phần mở rộng. Chương VII tổng kết đóng góp và nêu rõ các nội dung đã hoàn thiện so với bản giữa kỳ.

Điều nhóm chúng em thấy giá trị nhất sau quá trình này là: với các hệ detector hiện đại, hiểu bài báo thôi chưa đủ; cần chỉ ra được cấu hình nào, assigner nào, pipeline augmentation nào và script đánh giá nào đã biến ý tưởng trong paper thành hành vi quan sát được trong code. Chính bước nổi này làm rõ sự khác nhau giữa “đọc hiểu bài báo” và “thực sự tái hiện được phương pháp”.

Thành phố Hồ Chí Minh, ngày 19 tháng 4 năm 2026

Nhóm sinh viên thực hiện

Thông tin chung

Tên môn học:	Nhận dạng
Mã môn học:	CSC14006
Lớp:	23KHMT
Học kỳ:	2
Năm học:	2025–2026
Mã nguồn:	https://github.com/TQC0103/Nhan_dang
Giảng viên:	Thầy Võ Thanh Lâm Thầy Lê Hoàng Thái Thầy Nguyễn Thanh Tình
Sinh viên:	23127333 Trương Quốc Cường 23127011 Lê Anh Duy 23127199 Trần Gia Huy

Thông tin đề tài

- **Chủ đề:** phân tích và tái hiện bài báo *Sample and Computation Redistribution for Efficient Face Detection* (SCRFD).
- **Bài báo nền tảng:** *Sample and Computation Redistribution for Efficient Face Detection* của Jia Guo, Jiankang Deng, Alexandros Lattas và Stefanos Zafeiriou, công bố tại ICLR 2022. [1]
- **Mã nguồn dùng để tái hiện:** cây `detection/scrfd` của InsightFace, cùng các script thí nghiệm ở nhánh `experiments`. [2]
- **Dữ liệu:** WIDER FACE, dùng layout `data/retinaface/train` và `data/retinaface/val` với annotation `labelv2.txt`. [3]
- **Mục tiêu:** làm rõ động lực, phương pháp và đóng góp của SCRFD; chạy được pipeline official code cho SCRFD 2.5G; mô tả đầy đủ dataset/setup; chỉ ra mapping từ paper sang code; so sánh kết quả thu được với kết quả công bố; và trình bày phần mở rộng đủ chặt chẽ để đánh giá như một cải tiến kỹ thuật.
- **Phạm vi kế thừa từ giữa kỳ:** chỉ sử dụng phần Chương II liên quan trực tiếp đến bài báo SCRFD.
- **Phạm vi mở rộng:** *ASR+JSAR* là cải tiến chính trên official code; SPOS-NAS proxy chỉ là phân tích phụ về trực giác Computation Redistribution trong điều kiện tài nguyên giới hạn.
- **Đầu ra chính:** báo cáo kỹ thuật LaTeX, slide thuyết trình cuối kỳ, bảng so sánh WIDER FACE giữa baseline và phần mở rộng, cùng phần đối chiếu với kết quả mà SCRFD công bố.

Phân công thực hiện

Thành viên	Nội dung phụ trách
Trương Quốc Cường	Tổ chức lại cấu trúc report theo hướng báo cáo khoa học, chuẩn hóa template LaTeX, tổng hợp phần setup official reproduction, bảng đối chiếu với paper và pipeline build báo cáo.
Lê Anh Duy	Rà soát branch experiments , tổng hợp kết quả WIDER FACE, phân tích baseline, paper-SR12, <i>ASR+JSAR</i> , và viết phần so sánh kết quả thực nghiệm.
Trần Gia Huy	Đối chiếu paper với code, phân tích các module chính như search config, assigner, loss, evaluation và tóm tắt nhánh SPOS-NAS tài nguyên giới hạn như phần mở rộng.

Mục lục

Lời mở đầu	1
Thông tin chung	2
Thông tin đề tài	2
Phân công thực hiện	3
Danh mục hình	5
Danh mục bảng	6
Thuật ngữ và chữ viết tắt	7
I GIỚI THIỆU BÀI BÁO SCRFD	8
1 Bài toán và động lực nghiên cứu	8
2 Đóng góp chính của bài báo	8
3 Phạm vi của báo cáo này	9
II PHƯƠNG PHÁP SCRFD TRONG PAPER	10
1 Vị trí của SCRFD trong các detector một giai đoạn	10
2 Tổng quan kiến trúc Backbone–Neck–Head	10
3 Computation Redistribution	11
4 Sample Redistribution	13
5 Kết quả và nhận định trong paper gốc	14
6 Điểm mạnh, hạn chế và ý nghĩa thực tiễn	15
III TÁI HIỆN MÃ NGUỒN SCRFD CHÍNH THỨC	16
1 Nguồn tái hiện, cài đặt và dữ liệu	16
2 Cấu hình baseline chạy lại paper	17
3 Quy trình train và đánh giá	18
IV TỪ PAPER SANG CODE	19
1 Các thành phần hiện thực hóa ý tưởng chính	19
2 Lý do các lựa chọn kỹ thuật hợp lý	20
V KẾT QUẢ THỰC NGHIỆM VÀ SO SÁNH VỚI PAPER	21
1 Kết quả tái hiện baseline	21
2 Ảnh hưởng của paper-SR12 scale set	21
3 Giải thích độ lệch với kết quả công bố	22
VI MỞ RỘNG VÀ THẢO LUẬN	23
1 ASR+JSAR như cải tiến trên SCRFD	23
2 SPOS-NAS như cải tiến phân tích kiến trúc	27
VII KẾT LUẬN	33
Tài liệu tham khảo	34

Danh mục hình

II.1	Kiến trúc nền của SCRFD theo ba khối Backbone–Neck–Head.	10
II.2	Quy trình tìm kiếm hai bước trong Computation Redistribution.	13
VI.1	Minh họa luồng làm việc của SPOS (hình gốc từ bài báo SPOS): (a) Tại mỗi stage có nhiều "khối lựa chọn", trong mỗi lượt truyền (forward pass), supernet sẽ chọn ngẫu nhiên một khối để kích hoạt; (b) Cơ chế cắt trọng số (weight slicing) được kích hoạt khi hai khối ở hai tầng liên tiếp có kích thước in/out khác nhau, khối lớn hơn sẽ tự động cắt bớt số kênh cho tương thích; (c) Kỹ thuật mixed precision được tích hợp trong quá trình huấn luyện nhằm tối ưu bộ nhớ và thời gian.	27
VI.2	Đường cong hội tụ của supernet proxy; best validation loss đạt 0.6519 tại epoch 18.	28
VI.3	Phân bố validation loss của 1000 kiến trúc NAS trong thí nghiệm proxy tài nguyên giới hạn.	29
VI.4	Phân tích Funnel Principle: Top-100 winners đầu tư nhiều hơn vào C_2 và tránh bẫy tiêu tốn quá nhiều FLOPs ở C_3	30
VI.5	Phân bố độ rộng C_2 và C_5 : nhóm Top-100 tập trung quanh $C_2 \in \{40, 48, 56\}$ và tránh C_5 quá rộng.	30

Danh mục bảng

II.1	Các nhóm tham số chính trong không gian tìm kiếm Computation Redistribution.	12
II.2	Quy trình code-level để tìm kiến trúc theo Computation Redistribution.	12
II.3	Các bước Sample Redistribution trong pipeline huấn luyện.	14
III.1	Dữ liệu dùng để chạy lại SCRFD official.	16
III.2	Kiến trúc SCRFD-2.5GF dùng trong reproduction.	17
III.3	Hyperparameter và pipeline huấn luyện của baseline paper-SR12.	17
III.4	Protocol đánh giá WIDER FACE trong reproduction.	18
IV.1	Đối chiếu ý tưởng trong paper với thành phần triển khai tương ứng.	19
V.1	So sánh baseline tái hiện với số liệu <i>SCRFD-2.5GF</i> do model zoo chính thức công bố.	21
V.2	Ảnh hưởng của scale set lên baseline SCRFD 2.5G.	21
VI.1	Scale pool dùng trong thí nghiệm ASR+JSAR	24
VI.2	Kết quả WIDER FACE của ASR+JSAR trên hai scale set.	25
VI.3	Delta của ASR+JSAR so với baseline.	25
VI.4	JSAR positive assignment trước và sau fallback ở epoch cuối.	25
VI.5	Delta recall proxy trên hard subset theo kích thước mặt.	26
VI.6	Thống kê scale và face-size distribution sau SR simulation.	26
VI.7	Thiết lập chính của thí nghiệm SPOS-NAS proxy.	28
VI.8	Tóm tắt kết quả huấn luyện supernet proxy.	28
VI.9	Top-10 kiến trúc tốt nhất trong thí nghiệm SPOS-NAS.	29
VI.10	Phân bố chi phí tính toán giữa Top-100 winners và nhóm thua cùng mức GFLOPs.	30
VI.11	Dịch chuyển chi phí tính toán khi đổi proxy target từ center-focus sang edge-focus.	31

Thuật ngữ và chữ viết tắt

AMP

Automatic Mixed Precision, kỹ thuật huấn luyện hỗn hợp FP16/FP32 để tăng tốc và giảm bộ nhớ.

AP Average Precision, thước đo chính của bài toán phát hiện đối tượng và WIDER FACE.

ASR

Adaptive Sample Redistribution, mở rộng trong nhánh **experiments** dùng để cập nhật động phân phối scale augmentation.

ATSS

Adaptive Training Sample Selection, cơ chế assign positive/negative anchor được SCRFD sử dụng trong train.

BN Batch Normalization.

CR Computation Redistribution, ý tưởng tái phân bổ FLOPs vào các stage phù hợp hơn cho mặt nhỏ trong SCRFD.

FLOPs

Số phép toán dấu chấm động cần cho một lần suy luận.

GFLOPs

Tỷ đơn vị 10^9 FLOPs.

JSAR

Joint SampleAssignment Redistribution, phần mở rộng nói assignment cho tiny/small faces trong nhánh thí nghiệm.

NAS

Neural Architecture Search.

PAFPN

Path Aggregation Feature Pyramid Network.

QFL

Quality Focal Loss, hàm mất mát phân loại dùng trong **SCRFDHead**.

SCRFD

Sample and Computation Redistribution for Efficient Face Detection. [1]

SPOS

Single Path One-Shot, cách huấn luyện Supernet chia sẻ trọng số cho NAS. [4]

SR Sample Redistribution, cơ chế làm tăng tín hiệu huấn luyện cho các khuôn mặt nhỏ trong SCRFD.

WIDER FACE

Bộ dữ liệu benchmark phát hiện khuôn mặt ngoài trời, có chia Easy, Medium và Hard. [3]

Chương I

GIỚI THIỆU BÀI BÁO SCRFD

1 Bài toán và động lực nghiên cứu

Bài báo *Sample and Computation Redistribution for Efficient Face Detection* đặt mục tiêu xây dựng một họ detector khuôn mặt hiệu quả, có thể hoạt động ở nhiều mức ngân sách tính toán khác nhau nhưng vẫn giữ độ chính xác tốt trên các trường hợp khó của WIDER FACE. Trọng tâm của bài báo không nằm ở việc thêm một module phức tạp mới, mà nằm ở câu hỏi hệ thống hơn: với cùng một tổng số FLOPs, nên phân bổ năng lực tính toán và mẫu huấn luyện như thế nào để detector xử lý tốt khuôn mặt nhỏ, mờ, bị che khuất hoặc xuất hiện dày đặc. [1]

Trong WIDER FACE, split Hard thường là nơi thể hiện rõ nhất điểm yếu của các detector nhẹ. Khuôn mặt nhỏ chỉ chiếm vài pixel trên feature map sâu; nếu kiến trúc dồn quá nhiều tài nguyên vào tầng sâu hoặc nếu quá trình huấn luyện không tạo đủ mẫu dương ở thang nhỏ, detector có thể vẫn đạt điểm tốt trên Easy/Medium nhưng tụt mạnh trên Hard. SCRFD vì vậy xem face detection hiệu quả như một bài toán tối ưu đồng thời giữa *độ chính xác, chi phí suy luận và phân bố tín hiệu học*.

Với tập các stage backbone $\mathcal{S} = \{C_2, C_3, C_4, C_5\}$, tổng chi phí suy luận có thể viết:

$$\Phi_{\text{total}} = \sum_{s \in \mathcal{S}} \Phi_s(w_s, d_s, H_s, W_s),$$

trong đó w_s là số kênh, d_s là số block và $H_s \times W_s$ là kích thước không gian ở stage s . Trực giác quan trọng của SCRFD là *tổng FLOPs* không đủ để giải thích chất lượng detector; vị trí đặt FLOPs trong backbone, neck và head mới quyết định detector có giữ được tín hiệu khuôn mặt nhỏ hay không.

2 Đóng góp chính của bài báo

SCRFD đề xuất hai ý tưởng bổ trợ nhau. *Computation Redistribution* tái phân bổ chi phí tính toán giữa backbone, neck, head và giữa các stage trong backbone để tìm kiến trúc tốt hơn dưới cùng ngân sách FLOPs. *Sample Redistribution* điều chỉnh phân bố scale augmentation để tăng hiệu quả huấn luyện cho các vùng kích thước quan trọng, đặc biệt là các khuôn mặt rất nhỏ.

Bài báo cũng xây dựng một họ mô hình SCRFD từ mức rất nhỏ đến mức lớn hơn. Trong báo cáo này, nhóm tập trung vào miền *SCRFD-2.5GF* vì đây là điểm cân bằng phù hợp giữa chi phí tái hiện và khả năng quan sát tác động của hai cơ chế chính. Theo số liệu model zoo của InsightFace, *SCRFD-2.5GF* đạt 93.78/92.16/77.87 trên Easy/Medium/Hard, trong khi *ResNet-2.5GF* cùng mức FLOPs chỉ đạt 93.21/91.11/74.47. Khoảng chênh +3.40 điểm Hard AP cho thấy lợi ích của cách phân bổ tính toán, không chỉ lợi ích của việc tăng tổng dung lượng mạng. [2]

3 Phạm vi của báo cáo này

Báo cáo cuối kỳ này giữ SCRFD là đối tượng trung tâm và giới hạn phạm vi vào đúng một bài báo khoa học cụ thể, thay vì mở rộng sang các nội dung face detection tổng quát. Trong phạm vi đó, nhóm tập trung vào một cấu hình đại diện là *SCRFD-2.5GF* để trình bày trọn vẹn pipeline của paper: kiến trúc Backbone–Neck–Head, bước *Computation Redistribution* để tìm cấu hình, bước *Sample Redistribution* để điều chỉnh phân bố scale khi huấn luyện, và protocol đánh giá trên WIDER FACE.

Trên nền phạm vi đó, báo cáo đi theo ba lớp nội dung chính. Thứ nhất, trình bày lại SCRFD như một technical report hoàn chỉnh thay vì chỉ dừng ở mức tóm tắt ý tưởng. Thứ hai, tái hiện official code và đối chiếu kết quả với paper/model zoo để chỉ ra mức độ khớp và nguyên nhân chênh lệch. Thứ ba, phân tích hai hướng mở rộng bám trực tiếp vào pipeline này: *SPOS-NAS* như một góc nhìn cải tiến/phân tích cho bước *Computation Redistribution*, và *ASR+JSAR* như cải tiến cho *Sample Redistribution* cùng cơ chế assignment trong giai đoạn huấn luyện.

Chương II

PHƯƠNG PHÁP SCRFD TRONG PAPER

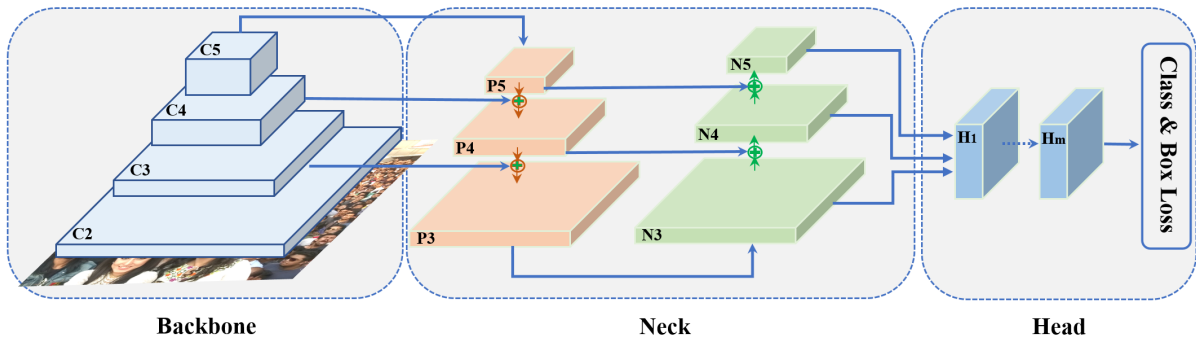
1 Vị trí của SCRFD trong các detector một giai đoạn

SCRFD được xây dựng trên nền tảng của các detector một giai đoạn dự đoán dày đặc. Các mô hình như SSD, YOLO và RetinaNet đã cho thấy có thể gộp bước sinh ứng viên và chấm điểm vào một lượt suy luận, đổi lại mô hình phải xử lý rất nhiều vị trí nền và anchor âm. RetinaNet giải quyết một phần mất cân bằng này bằng Focal Loss, còn RetinaFace đưa khung một giai đoạn vào bài toán khuôn mặt với thiết kế đa mức chuyên biệt hơn. [5], [6], [7], [8]

SCRFD đi tiếp theo một hướng khác: thay vì giới thiệu một head hoàn toàn mới, bài báo hỏi liệu detector hiện có đã dùng đúng tài nguyên hay chưa. Với ảnh VGA hoặc các cảnh đông người, khuôn mặt rất nhỏ phụ thuộc nhiều vào feature map độ phân giải cao. Do đó, thiết kế kế thừa trực tiếp từ backbone phân loại ảnh thường chưa tối ưu: các tầng sâu nhận nhiều compute hơn, trong khi các tầng nông quan trọng cho định vị mặt nhỏ lại có thể bị thiếu năng lực biểu diễn.

2 Tổng quan kiến trúc Backbone–Neck–Head

SCRFD là một one-stage anchor-based face detector gồm ba khối chính: *Backbone*, *Neck* và *Head*. Backbone trích xuất đặc trưng phân tầng từ ảnh đầu vào; neck dùng PAFPN để hợp nhất thông tin đa tỉ lệ; head dự đoán điểm khuôn mặt và hộp bao trên các mức feature map. Hình II.1 tóm tắt cấu trúc này.



Hình II.1: Kiến trúc nền của SCRFD theo ba khối Backbone–Neck–Head.

2.1 Backbone và các mức đặc trưng

Backbone tạo ra chuỗi đặc trưng $C_2 \rightarrow C_5$, trong đó các mức nông có stride nhỏ và giữ nhiều thông tin không gian hơn. Với đầu vào 640×640 , C_2 tương ứng khoảng 160×160 , C_3 khoảng 80×80 , còn các mức sâu hơn giảm nhanh độ phân giải. Đây là lý do SCRFD không xem mọi stage như nhau: khuôn mặt nhỏ cần tín hiệu định vị ở các tầng nông, trong khi tầng sâu chủ yếu bổ sung ngữ nghĩa.

Điểm quan trọng là các stage nông cũng là nơi chi phí tích chập đắt nhất do feature map lớn. Tăng width/depth ở C_2 hoặc C_3 có thể giúp tiny faces nhưng cũng làm FLOPs tăng mạnh. Vì vậy, bài toán của SCRFD không phải là “tăng tầng nông càng nhiều càng tốt”, mà là tìm tỉ lệ phân bổ hợp lý dưới ràng buộc ngân sách.

2.2 Neck PAFPN

Neck của SCRFD sử dụng PAFPN để kết hợp đặc trưng theo cả hướng trên-xuống và dưới-lên. Hướng trên-xuống truyền ngữ nghĩa từ tầng sâu về tầng nông; hướng dưới-lên giúp củng cố thông tin định vị và cấu trúc không gian. Cơ chế này phù hợp với face detection vì khuôn mặt xuất hiện ở nhiều thang đo, từ mặt lớn rõ nét đến mặt rất nhỏ chỉ chiếm vài pixel.

Trong Computation Redistribution, neck cũng nằm trong không gian cần cân bằng. Nếu neck quá rộng, FLOPs bị tiêu tốn ở khâu hợp nhất đặc trưng; nếu quá hẹp, thông tin từ backbone không được truyền hiệu quả đến head. SCRFD vì vậy không chỉ search backbone mà còn xét số kênh neck như một phần của toàn bộ detector.

2.3 Head, anchor và hàm mất mát

Head của SCRFD thực hiện hai nhiệm vụ song song: phân loại khuôn mặt/nền và hồi quy hộp bao. Thiết kế head được giữ gọn để chi phí dự đoán dày đặc không lấn át ngân sách của backbone. Với khuôn mặt, anchor cũng có thể đơn giản hơn detector vật thể tổng quát vì tỉ lệ hình học của khuôn mặt ít đa dạng hơn; paper ưu tiên anchor gần vuông và gắn kích thước anchor với từng mức stride.

Về loss, SCRFD kế thừa tinh thần xử lý mất cân bằng từ các detector dày đặc: nhánh classification dùng biến thể focal-style để giảm tác động của mẫu âm dễ, còn nhánh box dùng loss định vị như DIOU trong triển khai official. Điều này giải thích vì sao phần đối chiếu triển khai ở Chương IV cần kiểm tra cả head dự đoán, loss classification, loss bbox và cơ chế gán mẫu dương, chứ không chỉ nhìn vào backbone.

3 Computation Redistribution

Computation Redistribution là đóng góp trung tâm của SCRFD. Ý tưởng chính là tìm cách phân phối FLOPs giữa backbone, neck, head và giữa các stage trong backbone sao cho hiệu quả nhất cho face detection. Nếu chỉ giữ tổng FLOPs cố định mà không kiểm soát vị trí đặt FLOPs, hai detector cùng 2.5 GFLOPs có thể cho AP rất khác nhau trên Hard split.

Bài báo giảm bậc tự do của không gian tìm kiếm để tránh bùng nổ tổ hợp. Với backbone, mỗi stage C_i có hai nhóm tham số chính là số block d_i và độ rộng w_i . Stem được gộp vào C_2 , nên backbone còn 8 bậc tự do. Với neck và head, tác giả tiếp tục rút gọn bằng cách dùng số kênh neck n , số lớp head m và số kênh head h . Tổng thể, CR biến việc thiết kế detector thành một bài toán search có kiểm soát thay vì tinh chỉnh thủ công từng stage. [1]

Bảng II.1: Các nhóm tham số chính trong không gian tìm kiếm Computation Redistribution.

Thành phần	Tham số search	Ý nghĩa
Backbone	d_i, w_i với $i \in \{2, 3, 4, 5\}$	Điều chỉnh độ sâu và độ rộng từng stage để tái phân bổ compute theo độ phân giải feature map.
Neck	n	Chọn số kênh hợp nhất đặc trưng trong PAFPN, cân bằng giữa truyền thông tin đa tỉ lệ và chi phí.
Head	m, h	Chọn số lớp và số kênh head dự đoán dày đặc, tránh để head trở thành điểm tiêu FLOPs quá lớn.

Quy trình search của SCRFD được tổ chức theo hai bước. Bước đầu tập trung vào backbone để tìm template tốt theo Hard AP. Bước sau mở rộng search sang toàn mạng, gồm backbone, neck và head. Cách làm này giúp giảm chi phí tìm kiếm, đồng thời vẫn giữ được tính hệ thống của việc tối ưu toàn detector.

3.1 CR ở mức code: sinh cấu hình, lọc FLOPs và chọn kiến trúc

Trong mã nguồn SCRFD, Computation Redistribution không phải là một khẩu hiệu trừu tượng mà được hiện thực thành một pipeline sinh-lọc-huấn luyện-đánh giá cấu hình. File trung tâm là `search_tools/generate_configs_2.5g.py`. Script này nhận một config template, lấy mẫu ngẫu nhiên các tham số kiến trúc, thay chúng vào config detector, tính FLOPs bằng `get_model_complexity_info`, rồi chỉ ghi ra những config nằm trong khoảng FLOPs mục tiêu. Với mức 2.5GF, điều kiện lọc là:

$$2.5(1 - \epsilon) \leq \text{FLOPs}(\mathcal{A}) \leq 2.5(1 + \epsilon),$$

trong đó ϵ mặc định là 2×10^{-2} . Nghĩa là quá trình search không so sánh những mạng có ngân sách quá khác nhau; nó ép các candidate vào cùng vùng chi phí rồi để WIDER FACE AP quyết định cách phân bổ compute nào tốt hơn.

Bảng II.2: Quy trình code-level để tìm kiến trúc theo Computation Redistribution.

Bước	Thành phần code	Ý nghĩa kỹ thuật
1	<code>GenConfigBackbone</code>	Lấy mẫu block type, base channels, số block từng stage và tỉ lệ channel giữa các stage. Đây là pha search backbone.
2	<code>merge_cfg(det_cfg)</code>	Ghi các tham số đã lấy mẫu vào <code>det_cfg.model.backbone</code> , đồng thời cập nhật <code>neck.in_channels</code> để neck khớp với backbone mới.
3	<code>get_flops(cfg, input_shape)</code>	Build detector, chạy <code>forward_dummy</code> , đo FLOPs tổng và FLOPs theo nhóm: stem/layer1–layer4, neck, bbox head.
4	<code>is_flops_valid</code>	Chỉ giữ candidate có FLOPs gần 2.5GF trong sai số cho phép, tránh việc mô hình thắng chỉ vì dùng nhiều compute hơn.
5	<code>det_cfg.dump(output_cfg)</code>	Ghi candidate hợp lệ thành config mới như <code>configs/scrfdgen2.5g/scrfdgen2.5g_i.py</code> .
6	<code>search_train.sh</code> và <code>search_test.sh</code>	Train từng candidate, sau đó chạy <code>tools/test_widerface.py</code> để lấy AP Easy/Medium/Hard.

Pha search thứ nhất dùng `--mode 1`. Khi đó `GenConfigBackbone` thay đổi chủ yếu backbone: block type, base channels, số block từng stage và tỉ lệ channel giữa các stage. Sau khi train/evaluate các config được sinh ra, nhóm tác giả chọn một template backbone tốt làm điểm xuất phát. Pha thứ hai dùng `--mode 2`; lúc này `GenConfigAll` mở rộng search sang toàn detector bằng năm biến: tỉ lệ số block, tỉ lệ base channels, số kênh FPN, số kênh head và số lớp head. Trong mode này, script còn giữ tương đối tỉ lệ FLOPs giữa các stage backbone so với template đã chọn, rồi mới cho phép neck/head thay đổi. Nhờ vậy quá trình search toàn mạng không phá vỡ hoàn toàn backbone template đã tốt ở bước một.

Điểm quan trọng khi đọc kết quả CR là mỗi config candidate sau khi được sinh ra đều là một detector đầy đủ, không phải chỉ là một backbone rời. Candidate được train bằng script search, sinh checkpoint riêng trong `work\_dirs`, rồi đánh giá bằng WIDER FACE. Kết quả cuối cùng như SCRFD-2.5GF vì vậy là kết quả của quy trình chọn kiến trúc dựa trên AP dưới ràng buộc FLOPs, chứ không phải do tác giả thủ công đặt vài channel cho đẹp.



Hình II.2: Quy trình tìm kiếm hai bước trong Computation Redistribution.

Kết quả quan trọng của CR là các kiến trúc tối ưu không phân bổ FLOPs theo tỷ lệ backbone phân loại ảnh truyền thống. Chúng thường giữ năng lực tốt hơn ở tầng nông và tiết chế các thành phần ít hiệu quả hơn đối với tiny faces. Nhờ đó, SCRFD-2.5GF có thể vượt ResNet-2.5GF rõ nhất trên Hard AP dù cùng mức tính toán.

4 Sample Redistribution

Sample Redistribution giải quyết phía dữ liệu của cùng một vấn đề. Trong huấn luyện detector dày đặc, không phải mọi scale augmentation đều tạo ra cùng lượng mẫu dương hữu ích. Nếu scale làm khuôn mặt quá nhỏ so với anchor hoặc rơi vào mức feature không phù hợp, detector sẽ nhận tín hiệu học yếu hơn. Ngược lại, một phân bố scale tốt có thể tăng mật độ positive samples cho các vùng kích thước quan trọng mà không làm tăng chi phí suy luận.

SCRFD tìm kiếm một tập scale augmentation để cải thiện độ phủ của tiny faces. Cơ chế này đặc biệt hiệu quả khi đi cùng CR: kiến trúc đã dành nhiều tài nguyên hơn cho feature map nông, còn SR đảm bảo trong quá trình train các feature map đó nhận đủ mẫu dương có ý nghĩa. Nói cách khác, CR tối ưu *năng lực mô hình*, còn SR tối ưu *cách dữ liệu kích hoạt năng lực đó*.

Từ góc nhìn triển khai, SR chủ yếu xuất hiện qua pipeline crop/resize và các cấu hình scale set. Trong report cuối kỳ, nhóm dùng paper-SR12 để đưa baseline gần hơn với thiết lập của paper. Kết quả ở Chương V cho thấy chỉ riêng việc chuyển từ default scale set sang paper-SR12 đã tăng Hard AP từ 72.49 lên 73.71, củng cố vai trò của phân bố scale trong SCRFD.

4.1 SR ở mức code: scale set đi qua pipeline huấn luyện như thế nào

Trong code, Sample Redistribution đi vào mô hình qua transform `RandomSquareCrop` trong `mmdet/datasets/pipelines/transforms.py`. Transform này nhận một danh sách `crop\_choice`; mỗi lần đọc một ảnh train, nó lấy một scale từ danh sách đó, tạo crop vuông có cạnh:

$$c = \lfloor s \cdot \min(W, H) \rfloor,$$

với s là scale được chọn, W, H là kích thước ảnh gốc. Sau đó crop được resize về 640×640 . Vì vậy $s < 1$ thường tạo hiệu ứng zoom-in, còn $s > 1$ tạo hiệu ứng zoom-out. Đây là chỗ SR tác động trực tiếp vào phân bố kích thước khuôn mặt sau augmentation.

Bảng II.3: Các bước Sample Redistribution trong pipeline huấn luyện.

Bước	Thành phần code	Tác dụng
1	<code>LoadImageFromFile, LoadAnnotations</code>	Nạp ảnh, bbox và keypoint theo định dạng RetinaFace/SCRFD.
2	<code>RandomSquareCrop(crop_choice=...)</code>	Chọn một scale từ scale set, crop ảnh thành patch vuông, giữ các bbox có tâm nằm trong patch.
3	<code>bbox_clip_border=False</code>	Không đơn giản cắt mọi bbox vào biên theo cách làm mất ý nghĩa hình học của crop; pipeline giữ logic riêng của SCRFD khi xử lý bbox/keypoint.
4	<code>Resize(img_scale=(640,640), keep_ratio=False)</code>	Đưa mọi crop về cùng input size để detector train ổn định. Scale đã chọn quyết định mặt trong crop trở nên lớn hay nhỏ sau resize.
5	<code>RandomFlip, PhotoMetricDistortion, Normalize</code>	Bổ sung nhiễu hình học/ánh sáng, rồi chuẩn hóa ảnh với mean 127.5, std 128.0.
6	ATSSAssigner trong head	Sau augmentation, các bbox mới được gán positive/negative anchors; scale set ảnh hưởng gián tiếp đến số positive theo kích thước mặt.

Default source scale set của SCRFD-2.5GF dùng $[0.3, 0.45, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0]$. Trong reproduction cuối kỳ, nhóm dùng paper-SR12 gồm 12 scale trong khoảng $0.5 \rightarrow 2.6$. Sự khác biệt không chỉ là thêm vài giá trị: paper-SR12 có nhiều zoom-out mạnh hơn, nên tạo ra nhiều tình huống mặt nhỏ hơn sau resize. Đây là lý do cùng một kiến trúc SCRFD-2.5GF nhưng đổi scale set đã làm Hard AP tăng từ 72.49 lên 73.71 trong thí nghiệm của nhóm.

Nói cách khác, SR có thể được đọc như một vòng nhân quả rõ ràng ở mức code: `crop\_choice` quyết định scale crop; scale crop quyết định kích thước bbox sau resize; kích thước bbox quyết định anchors/feature levels nào nhận positive samples; cuối cùng lượng supervision ở nhóm tiny/small ảnh hưởng trực tiếp đến AP trên Hard split. Phần ASR+JSAR ở Chương VI mở rộng đúng đường nhân quả này: ASR thay `crop\_choice` tính bằng phân phối scale động, còn JSAR sửa bước assignment để tiny faces không bị thiếu positive anchors.

5 Kết quả và nhận định trong paper gốc

Paper SCRFD báo cáo kết quả trên WIDER FACE theo ba split Easy/Medium/Hard, đồng thời so sánh với các detector cùng hoặc khác mức chi phí. Một điểm nổi bật là SCRFD cải thiện mạnh ở Hard, tức vùng có nhiều khuôn mặt nhỏ, đông và khó. Ở chế độ VGA, paper cho thấy SCRFD-34GF đạt Hard AP 86.21, cao hơn TinaFace 81.43 trong khi chi phí tính toán thấp hơn đáng kể; cùng lúc, các cấu hình nhẹ như SCRFD-0.5GF vẫn giữ được lợi thế so với các baseline MobileNet-like trên Hard set. [1]

Những kết quả này cho thấy đóng góp của SCRFD không chỉ nằm ở một con số AP đơn lẻ, mà nằm ở đường cong đánh đổi accuracy–efficiency. Họ mô hình SCRFD tạo ra nhiều điểm làm việc khác nhau: cấu hình nhỏ cho thiết bị biên, cấu hình 2.5GF cho cân bằng chi phí/chất lượng, và cấu hình lớn hơn cho tình huống cần độ chính xác cao. Đây là lý do báo cáo cuối kỳ chọn 2.5GF làm miền tái hiện chính: nó đủ nhẹ để kiểm chứng trong điều kiện tài nguyên nhóm, nhưng vẫn đủ nhạy để quan sát tác động của CR và SR.

6 Điểm mạnh, hạn chế và ý nghĩa thực tiễn

Điểm mạnh lớn nhất của SCRFD là tư duy tối ưu ở cấp hệ thống. Paper không chỉ sửa một loss hoặc thêm một block, mà xem detector như một phân phối tài nguyên cần được thiết kế lại theo đặc thù face detection. CR xử lý phân bổ compute, SR xử lý phân bổ mẫu huấn luyện, còn kết quả WIDER FACE chứng minh cả hai cơ chế cùng tác động mạnh nhất vào Hard split.

Hạn chế chính nằm ở chi phí tìm kiếm kiến trúc và khả năng chuyển miền. Pha empirical bootstrap/search trong paper cần tài nguyên lớn, nên các nhóm nhỏ khó tái lập đầy đủ từ đầu. Ngoài ra, cấu hình tối ưu phụ thuộc vào WIDER FACE, độ phân giải mục tiêu và thiết kế detector nền; khi chuyển sang camera, domain hoặc tác vụ khác, có thể cần tinh chỉnh lại scale augmentation, threshold, hoặc thậm chí không gian search.

Về ứng dụng, SCRFD đưa ra một thông điệp rất thực dụng: không nên lấy nguyên backbone phân loại ảnh rồi áp vào phát hiện khuôn mặt nhỏ. Nếu ngân sách tính toán cố định, cách phân bổ compute và cách tạo mẫu huấn luyện có thể quan trọng không kém tổng dung lượng mô hình. Đây cũng là cơ sở cho phần mở rộng ASR+JSAR ở Chương VI: thay vì tăng chi phí suy luận, nhóm thử cải thiện cách sample và assignment tạo supervision cho nhóm mặt nhỏ.

Chương III

TÁI HIỆN MÃ NGUỒN SCRFD CHÍNH THỨC

1 Nguồn tái hiện, cài đặt và dữ liệu

Phần tái hiện chính sử dụng triển khai SCRFD trong dự án InsightFace thay vì viết lại detector từ đầu. Mục tiêu của bước này là kiểm tra nhóm có thể cài đặt, train và evaluate pipeline SCRFD theo cách gần với tác giả nhất trong điều kiện tài nguyên hiện có. Vì vậy, reproduction được giữ tách biệt với phần cải tiến: trước hết chạy baseline SCRFD-2.5GF, sau đó mới thay đổi sample/assignment ở Chương VI.

Quy trình cài đặt đi theo README của SCRFD/InsightFace: cài `mmcv-full` tương thích với nhánh MMDetection cũ mà SCRFD sử dụng, cài build requirements, sau đó cài package detection ở chế độ editable bằng `pip install -v -e ..` README của tác giả ghi nhận `mmcv-full==1.2.6` và `1.3.3` là các phiên bản đã được kiểm thử. Đây là điểm quan trọng vì SCRFD không chỉ là một file model độc lập; nó phụ thuộc vào registry, dataset, pipeline, assigner, loss và evaluation script của MMDetection.

Dataset được chuẩn bị theo layout `RetinaFaceDataset` mà official SCRFD yêu cầu:

Bảng III.1: Dữ liệu dùng để chạy lại SCRFD official.

Mục	Thiết lập
Dataset	WIDER FACE train và validation
Định dạng đọc dữ liệu	<code>RetinaFaceDataset</code> của SCRFD/MMDetection
Thư mục train	<code>data/retinaface/train/images/</code>
Thư mục validation/test	<code>data/retinaface/val/images/</code>
Annotation train	<code>data/retinaface/train/labelv2.txt</code>
Annotation validation/test	<code>data/retinaface/val/labelv2.txt</code>
Số ảnh hợp lệ dùng trong thống kê	Khoảng 12 876 ảnh train và 3 222 ảnh validation sau bước chuẩn bị dữ liệu
Task đánh giá	WIDER FACE Easy, Medium, Hard

Các annotation dùng cả bounding box và keypoint theo định dạng RetinaFace/SCRFD. Tuy nhiên, cấu hình SCRFD-2.5GF trong reproduction đặt `use_kps=False`, nên keypoint được nạp theo pipeline nhưng loss chính vẫn là classification và bbox regression. Điều này giúp reproduction bám đúng detector face box của paper thay vì biến thí nghiệm thành bài toán landmark.

2 Cấu hình baseline chạy lại paper

Baseline chính là SCRFD-2.5GF train 80 epoch, dùng cosine scheduler và paper-SR12 scale set. Đây là thiết lập được chọn vì nó đủ gần tinh thần Sample Redistribution của paper hơn scale set mặc định trong source, đồng thời vẫn nằm trong chi phí có thể chạy lại. Bảng dưới đây ghi rõ các thành phần kiến trúc thay vì chỉ ghi chung chung “SCRFD-2.5G”.

Bảng III.2: Kiến trúc SCRFD-2.5GF dùng trong reproduction.

Thành phần	Cấu hình cụ thể
Backbone	<code>ResNetV1e</code> , <code>BasicBlock</code> , stage blocks (3, 5, 3, 2), stage channels [24, 48, 48, 80]
Neck	<code>PAFPN</code> , input channels [24, 48, 48, 80], output channel 24, <code>start_level=1</code> , <code>num_outs=3</code>
Detection head	<code>SCRFDHead</code> , 1 class, 2 stacked convs, feature channel 64, shared cls/reg branch
Normalization trong head	<code>GroupNorm</code> , 16 groups
Anchor generator	Ratio 1.0, scales [1, 2], base sizes [16, 64, 256], strides [8, 16, 32]
Classification loss	<code>QualityFocalLoss</code> , sigmoid, $\beta = 2.0$, weight 1.0
BBox loss	<code>DIoULoss</code> , weight 2.0
Assigner	<code>ATSSAssigner</code> , <code>topk=9</code> , <code>mode=0</code>

Bảng III.3: Hyperparameter và pipeline huấn luyện của baseline paper-SR12.

Mục	Giá trị
Input train	Crop vuông ngẫu nhiên rồi resize cố định về 640×640 , <code>keep_ratio=False</code>
Input test	Resize giữ tỉ lệ, pad về 640×640 , không flip test-time
Số epoch	80
Optimizer	SGD, learning rate 0.01, momentum 0.9, weight decay 5×10^{-4}
Scheduler	Cosine annealing, warmup tuyến tính 1500 iterations, warmup ratio 0.001, minimum LR 0
Batch size	8 ảnh/GPU theo cấu hình <code>samples_per_gpu</code>
Data workers	2 workers/GPU
Augmentation	<code>RandomSquareCrop</code> , <code>RandomFlip</code> 0.5, <code>PhotoMetricDistortion</code> , normalize mean 127.5, std 128.0
Paper-SR12 scale set	12 scale trong khoảng $0.5 \rightarrow 2.6$
Checkpoint/evaluation interval	Lưu và đánh giá ở epoch 80

Với `RandomSquareCrop`, ý nghĩa của scale không phải chỉ là phóng to/thu nhỏ ảnh trực tiếp. Scale điều khiển kích thước crop trước khi ảnh được resize về 640×640 . Scale nhỏ tương ứng zoom-in, làm mất trong crop lớn hơn; scale lớn tương ứng zoom-out, làm mất nhỏ hơn và tạo thêm tình huống tiny/dense. Vì vậy paper-SR12 có vai trò rất gần với Sample Redistribution trong paper: nó thay đổi phân bố kích thước khuôn mặt mà detector nhìn thấy trong quá trình học.

3 Quy trình train và đánh giá

Sau khi dữ liệu và môi trường được chuẩn bị, nhóm chạy baseline bằng config paper-SR12 cosine:

```
configs/scrfd/scrfd_2.5g_80e_cosine_baseline_paper_sr12.py.
```

Về mặt thao tác, quy trình gồm hai lệnh loại chuẩn của SCRFD/MMDetection: train bằng script distributed training của repo, sau đó evaluate bằng `tools/test\_widerface.py` với checkpoint cuối cùng. Khi chỉ có một GPU, distributed script vẫn có thể chạy với world size 1; khi có nhiều GPU, batch tổng tăng tuyến tính theo số GPU theo cấu hình `samples\_per\_gpu`.

Khi đánh giá, nhóm dùng đúng protocol WIDER FACE của SCRFD:

Bảng III.4: Protocol đánh giá WIDER FACE trong reproduction.

Mục	Giá trị
Script đánh giá	<code>tools/test_widerface.py</code>
Input validation	<code>data/retinaface/val/images/</code> và <code>val/labelv2.txt</code>
Score threshold	0.02
NMS	NMS IoU 0.45
Giới hạn số bbox mỗi ảnh	<code>max_per_img=-1</code>
Output chính	AP trên Easy, Medium, Hard
Checkpoint baseline	<code>work_dirs/compare_2.5g_paper_sr12_cosine/baseline/latest.pth</code>
Thời điểm đánh giá baseline	2026-04-18 19:29:05

Bằng chứng chạy thành công không chỉ là có một con số AP cuối cùng. Kết quả lưu lại config đã dùng, checkpoint, timestamp đánh giá, threshold test và AP từng split. Baseline paper-SR12 thu được 91.61/89.99/73.71 trên Easy/Medium/Hard. Ngoài baseline chính, nhóm còn chạy thêm baseline với scale set mặc định của source để kiểm tra riêng vai trò của scale redistribution; phần so sánh này được trình bày ở Chương V.

Về phần cứng, các thí nghiệm được tổ chức trên môi trường GPU CUDA để train SCRFD trong MMDetection. Tuy nhiên log kết quả hiện có không ghi fingerprint GPU cụ thể cho từng run, nên báo cáo không gán cứng một model GPU cho reproduction SCRFD. Phần chắc chắn và có thể kiểm chứng là cấu hình phần mềm, dataset layout, hyperparameter, checkpoint, timestamp và protocol đánh giá. Đây là cách báo cáo trung thực hơn so với việc điền một tên GPU không được log lại trong artifact.

Chương IV

TỪ PAPER SANG CODE

1 Các thành phần hiện thực hóa ý tưởng chính

Để chứng minh đã hiểu phương pháp ở mức triển khai, báo cáo không chỉ mô tả SCRFD có CR và SR, mà còn chỉ ra các ý tưởng đó được hiện thực bởi những thành phần nào trong hệ thống huấn luyện–đánh giá. Bảng dưới đây đối chiếu các thành phần chính của paper với cơ chế triển khai tương ứng.

Bảng IV.1: Đối chiếu ý tưởng trong paper với thành phần triển khai tương ứng.

Ý tưởng trong paper	Thành phần triển khai	Vai trò	Nếu bỏ hoặc sửa
Computation Redistribution	Searched backbone 2.5G và quy trình chọn kiến trúc theo Hard AP	Cố định cách phân bổ FLOPs đã được tối ưu cho face detection.	Nếu thay bằng backbone phân bổ FLOPs kém hợp lý, Hard AP giảm; model zoo cho thấy ResNet-2.5GF thấp hơn SCRFD-2.5GF 3.40 điểm Hard.
Sample Redistribution	Pipeline crop/resize và paper-SR12 scale set	Điều khiển phân phối kích thước mặt mà mô hình thấy trong train.	Nếu scale set ít tiny-oriented hơn, baseline yếu đi trên Hard; paper-SR12 baseline cao hơn default baseline trong thí nghiệm.
Dense head và loss	Đầu dự đoán dày đặc của SCRFD	Hiện thực anchor generator, loss phân loại dạng focal-style, loss hồi quy hộp và projection từ đặc trưng sang box/classification.	Nếu thay đổi head hoặc loss không kiểm soát, cân bằng classification/localization lệch và AP giảm.
Assignment khi train	Cơ chế gán positive anchors động	Chọn positive anchors theo khoảng cách tâm và ngưỡng IoU động.	Nếu assignment không phù hợp, tiny faces nhận quá ít positive anchors và Hard AP bị ảnh hưởng đầu tiên.
Evaluation trên WIDER FACE	Protocol đánh giá Easy/Medium/Hard	Xuất AP Easy/Medium/Hard và tổng hợp delta giữa hai thiết lập.	Nếu không dùng đúng protocol eval, việc so sánh với số liệu WIDER FACE của paper/model zoo dễ thiếu công bằng.

2 Lý do các lựa chọn kỹ thuật hợp lý

Các lựa chọn kỹ thuật trong SCRFD đều hướng về mục tiêu giữ detector nhẹ nhưng không bỏ rơi khuôn mặt rất nhỏ. Backbone 2.5G được search để giữ năng lực biểu diễn ở vùng đầu mạng, vì tiny faces cần độ phân giải không gian tốt. Random square crop và scale set SR12 giúp mô hình thấy nhiều tình huống scale hơn trong train. Cơ chế assignment động làm việc chọn positive anchors ít phụ thuộc hơn vào một ngưỡng IoU cứng, còn head dự đoán của SCRFD được giữ đủ gọn để phù hợp mục tiêu efficient detection.

Nếu thay các thành phần này bằng lựa chọn đơn giản hơn, model thường không hỏng hoàn toàn, nhưng Hard split sẽ giảm trước. Đây là dấu hiệu hợp lý vì Hard chính là nơi phụ thuộc mạnh nhất vào mật độ positive samples, độ phân giải feature map và cách phân bổ compute.

Chương V

KẾT QUẢ THỰC NGHIỆM VÀ SO SÁNH VỚI PAPER

1 Kết quả tái hiện baseline

Bảng V.1: So sánh baseline tái hiện với số liệu *SCRFD-2.5GF* do model zoo chính thức công bố.

Thiết lập	Easy	Medium	Hard	Ghi chú
SCRFD-2.5GF official	93.78	92.16	77.87	Số liệu công bố trên model zoo
Baseline tái hiện (paper-SR12, cosine 80e)	91.61	89.99	73.71	Kết quả do nhóm chạy lại
Độ lệch so với official	-2.17	-2.17	-4.16	Gap lớn nhất nằm ở Hard

Nếu lấy trung bình ba split, baseline tái hiện đạt khoảng 85.10, trong khi giá trị suy ra từ số liệu official là khoảng 87.94. Chênh lệch khoảng 2.83 điểm là có thật, nhưng vẫn có thể giải thích trong bối cảnh reproduction: số official là kết quả model zoo hoàn chỉnh, còn thực nghiệm của nhóm ưu tiên khả năng lặp lại và so sánh kiểm soát giữa các thiết lập huấn luyện.

2 Ảnh hưởng của paper-SR12 scale set

Bảng V.2: Ảnh hưởng của scale set lên baseline SCRFD 2.5G.

Baseline	Easy	Medium	Hard
Default source scale set	91.40	89.69	72.49
Paper-SR12 scale set	91.61	89.99	73.71
Delta (paper-SR12 - default)	+0.21	+0.31	+1.22

Việc quay về đúng 12-scale SR set của paper giúp baseline tăng rõ nhất trên Hard. Điều này phù hợp với giả thuyết của SCRFD: scale augmentation tác động trực tiếp đến lượng tiny faces hữu ích trong huấn luyện, còn Hard split là nơi thay đổi đó thể hiện rõ nhất.

3 Giải thích độ lệch với kết quả công bố

Độ lệch giữa reproduction và official number có thể đến từ ba nguyên nhân chính. Thứ nhất, số liệu official phản ánh searched model và pipeline được tác giả tinh chỉnh đầy đủ, trong khi reproduction của nhóm chạy trong một khung train/eval kiểm soát để so sánh baseline và biến thể. Thứ hai, nhóm không tái tạo toàn bộ quá trình empirical bootstrap, search và chọn checkpoint của tác giả. Thứ ba, nhật ký kết quả không lưu fingerprint GPU cụ thể, nên phần mềm và hyperparameter có thể kiểm chứng tốt hơn phần môi trường phần cứng.

Điểm quan trọng là độ lệch lớn nhất nằm ở Hard, đúng với nơi SCRFD nhắm tới. Vì vậy, gap này không chỉ là một sai khác định lượng; nó cũng chỉ ra vùng mà pipeline tái hiện còn khó bắt kịp nhất: supervision và phân bổ compute cho khuôn mặt nhỏ trong các cảnh đông, mờ hoặc bị che khuất.

Chương VI

MỞ RỘNG VÀ THẢO LUẬN

1 ASR+JSAR như cải tiến trên SCRFD

ASR+JSAR là cải tiến trực tiếp trên SCRFD theo hướng thay đổi tín hiệu huấn luyện thay vì thay đổi kiến trúc suy luận. Điểm thiết kế quan trọng là backbone, neck, dense head và đường inference được giữ nguyên; cải tiến chỉ tác động vào hai nơi trong quá trình train: cách chọn scale augmentation và cách gán positive anchors cho khuôn mặt rất nhỏ. Vì vậy, nếu cải tiến có lợi, lợi ích đến từ việc mô hình nhận được supervision tốt hơn cho tiny/hard faces, không phải từ việc tăng FLOPs hoặc thêm module suy luận mới.

1.1 Động cơ thiết kế

SCRFD đã chỉ ra rằng Hard split chịu ảnh hưởng mạnh bởi mặt nhỏ, nhưng trong reproduction vẫn còn hai nguồn thiếu tín hiệu. Thứ nhất, scale augmentation tĩnh có thể không tạo ra đủ các trường hợp tiny/small phù hợp với giai đoạn học hiện tại của mô hình. Thứ hai, ngay cả khi ảnh có tiny faces, ATSS assignment gốc có thể gán quá ít positive anchors cho các ground-truth rất nhỏ vì IoU thấp và center constraint quá chặt. ASR+JSAR xử lý hai nút thắt này theo hai hướng bổ sung:

- **ASR (Adaptive Sample Redistribution):** học lại phân phối chọn crop scale từ thống kê difficulty theo size bin trong quá trình train.
- **JSAR (Joint Sample–Assignment Redistribution):** tăng mật độ positive anchors cho tiny/small faces bằng threshold relaxation, center-gating relaxation và fallback positives.

Nhóm chia ground-truth thành bốn bin theo kích thước $[0, 16)$, $[16, 32)$, $[32, 96)$, $[96, \infty)$, tương ứng tiny, small, medium và large. Các thống kê chính được theo dõi gồm số ground-truth G_b , số positive anchors P_b , classification loss L_b^{cls} , box loss L_b^{box} , positive histogram trước/sau JSAR và positive count theo từng FPN level. Những thống kê này vừa điều khiển ASR, vừa dùng để kiểm tra JSAR có thật sự tăng supervision cho tiny faces hay không.

1.2 Cơ chế Adaptive Sample Redistribution

Trong baseline, RandomSquareCrop lấy scale từ một danh sách cố định. ASR thay phân phối tĩnh đó bằng một phân phối động $p_t(s)$ trên các scale candidates. Ở mỗi giai đoạn cập nhật, mô hình ước lượng độ khó của từng size bin từ ba tín hiệu:

$$r_b = \max \left(0, 1 - \min \left(\frac{P_b}{\max(G_b, 1)}, 1 \right) \right),$$
$$c_b = \frac{L_b^{cls}}{\max(P_b, 1)}, \quad u_b = \frac{L_b^{box}}{\max(P_b, 1)}.$$

Ở chế độ *loss-recall*, difficulty của bin b được tính từ tổng các thành phần đã chuẩn hóa:

$$d_b \propto \text{norm}(r_b) + \text{norm}(c_b) + \text{norm}(u_b).$$

Difficulty theo bin sau đó được chiếu sang scale candidates thông qua một ma trận hỗ trợ $S(s, b)$. Trực giác hình học là scale lớn hơn tạo crop lớn hơn, sau resize về 640×640 sẽ làm mặt nhỏ hơn; scale nhỏ hơn tương ứng zoom-in, làm mặt lớn hơn. Vì vậy ASR không đơn giản “tăng tiny” một chiều, mà học cách phân phối xác suất giữa zoom-in và zoom-out theo độ khó quan sát được. Sau khi có phân phối thô $q(s)$, hệ thống áp probability floor để tránh collapse scale diversity, rồi dùng EMA:

$$p_{t+1}(s) = \alpha p_t(s) + (1 - \alpha)q'(s).$$

Trong các thí nghiệm cuối cùng, ASR dùng warm-up 2 epoch, cập nhật mỗi 1000 iterations, $\alpha = 0.8$, minimum probability 0.03, difficulty mode *loss-recall*. Hai scale pool được dùng để kiểm tra liệu cải thiện đến từ cơ chế redistribution hay chỉ đến từ một tập scale cố định:

Bảng VI.1: Scale pool dùng trong thí nghiệm ASR+JSAR

Thiết lập	Số scale	Khoảng scale	Ghi chú
Default source scale set	10	$0.35 \rightarrow 2.0$	Pool hẹp hơn, zoom-in mạnh hơn ở biên trái
Paper-SR12 scale set	12	$0.5 \rightarrow 2.6$	Gần hơn với policy scale của SCRFD paper

1.3 Cơ chế Joint Sample–Assignment Redistribution

JSAR được xây trên ATSS assignment. ATSS gốc chọn candidates theo khoảng cách tâm trên từng feature level, tính ngưỡng $\tau_g = \mu_g + \sigma_g$ từ IoU của candidates, rồi giữ anchors có IoU vượt ngưỡng và có center hợp lệ. Với tiny faces, IoU của anchor thường thấp và center constraint dễ loại bỏ nhiều anchors, nên một ground-truth có thể nhận quá ít positive samples.

JSAR thay đổi quá trình này ở ba bước. Trước hết, ngưỡng IoU được nối theo kích thước:

$$\tau'_g = \tau_g - \Delta_g,$$

trong đó $\Delta_g = 0.05$ cho tiny faces dưới 16 px, $\Delta_g = 0.02$ cho small faces từ 16 đến 32 px, và bằng 0 cho các mặt lớn hơn. Tiếp theo, center gating được nối theo hệ số size-aware với hệ số tối đa 1.3, giúp các anchors gần tiny GT không bị loại quá sớm. Cuối cùng, ở chế độ *hybrid fallback*, nếu một tiny/small GT vẫn có ít hơn số positives tối thiểu, JSAR thêm tối đa 4 anchors nền gần nhất theo score:

$$\text{score}(a, g) = \text{IoU}(a, g) - 0.05 \frac{\text{dist}(a, g)}{\max(\text{size}_g, 1)}.$$

Thiết lập chính dùng $N_{\min} = 3$ positives cho tiny GT. Với small GT, yêu cầu tối thiểu được giảm nhẹ để tránh ép quá nhiều anchors vào các trường hợp đã đủ supervision. Như vậy JSAR không phá toàn bộ behavior của ATSS; nó chỉ can thiệp vào vùng tiny/small, còn medium/large gần như giữ nguyên.

1.4 Protocol thực nghiệm

Cả baseline và ASR+JSAR đều dùng SCRFD-2.5GF, train 80 epoch, cosine scheduler, input 640×640 , đánh giá WIDER FACE Easy/Medium/Hard với score threshold 0.02 và NMS IoU 0.45. Nhóm chạy hai cặp thí nghiệm để tách ảnh hưởng của static scale pool:

- **Default source scale set:** baseline dùng scale pool mặc định của source; ASR+JSAR dùng adaptive policy trên pool tương ứng.

- **Paper-SR12 scale set:** baseline dùng 12 scale theo paper; ASR+JSAR dùng cùng pool SR12 nhưng cập nhật xác suất động.

Việc so sánh trên hai scale set là cần thiết vì paper-SR12 vốn đã tiny-oriented hơn default scale set. Nếu ASR+JSAR vẫn tăng Hard AP ở cả hai trường hợp, điều đó cho thấy cải tiến không chỉ ăn may nhờ một scale pool cụ thể.

Bảng VI.2: Kết quả WIDER FACE của ASR+JSAR trên hai scale set.

Thiết lập	Easy AP	Medium AP	Hard AP	mAP
Default baseline	91.40	89.69	72.49	84.53
Default ASR+JSAR	90.39	89.25	76.90	85.51
Paper-SR12 baseline	91.61	89.99	73.71	85.10
Paper-SR12 ASR+JSAR	90.28	89.01	77.38	85.56

Bảng VI.3: Delta của ASR+JSAR so với baseline.

Scale set	Easy	Medium	Hard	mAP
Default source	-1.00	-0.44	+4.40	+0.99
Paper-SR12	-1.33	-0.98	+3.67	+0.45

Mẫu hình kết quả rất rõ: ASR+JSAR không tối ưu đồng đều mọi split, mà tập trung vào Hard. Easy và Medium giảm nhẹ vì phương pháp chấp nhận dịch training signal về tiny/hard faces; đổi lại Hard AP tăng +4.40 điểm với default scale set và +3.67 điểm với paper-SR12. Với paper-SR12, Hard AP đạt 77.38, chỉ còn cách số official SCRFD-2.5GF 77.87 khoảng 0.49 điểm ở split khó nhất.

1.5 JSAR tăng supervision cho tiny faces

Bảng chứng trực tiếp nhất của JSAR nằm ở số positive anchors trên mỗi GT theo size bin. Ở default scale set, tiny positives/GT tăng từ 1.73 lên 2.67, tương ứng boost 1.54×. Ở paper-SR12, con số này tăng từ 1.76 lên 2.67, tương ứng boost 1.52×. Trong khi đó, small/medium/large gần như không đổi: small giữ khoảng 6.02, medium khoảng 5.45, large khoảng 5.05. Điều này đúng với mục tiêu thiết kế: JSAR không làm assignment trở nên lỏng lẻo cho mọi kích thước, mà chỉ bù supervision cho tiny faces.

Bảng VI.4: JSAR positive assignment trước và sau fallback ở epoch cuối.

Scale set	Bin	Before / GT	After / GT	Boost
Default source	Tiny	1.73	2.67	1.54×
Default source	Small	6.02	6.02	1.00×
Paper-SR12	Tiny	1.76	2.67	1.52×
Paper-SR12	Small	6.04	6.04	1.00×

Từ góc nhìn cơ chế, đây là kết quả quan trọng hơn việc chỉ nhìn AP. Nó chứng minh phần code JSAR thực sự tác động đúng vào vùng cần tác động: tiny faces vốn bị under-assigned được tăng số positives, còn các bin lớn không bị thay đổi đáng kể. Điều này cũng giải thích vì sao Hard AP tăng mạnh hơn Easy/Medium.

1.6 Hard subset: gain tập trung ở mặt rất nhỏ

Phân tích hard subset cho thấy gain recall proxy tập trung gần như hoàn toàn vào hai bin nhỏ nhất. Với default scale set, recall proxy tăng +0.1385 ở bin [0, 8) và +0.1190 ở bin [8, 16); bin [16, 32) chỉ tăng +0.0089, còn các

bin lớn gần như không tăng hoặc giảm rất nhẹ. Với paper-SR12, mô hình tương tự: $[0, 8)$ tăng $+0.1077$, $[8, 16)$ tăng $+0.1040$, $[16, 32)$ chỉ tăng $+0.0032$.

Bảng VI.5: Delta recall proxy trên hard subset theo kích thước mặt.

Size bin	Default source	Paper-SR12
$[0, 8)$	$+0.1385$	$+0.1077$
$[8, 16)$	$+0.1190$	$+0.1040$
$[16, 32)$	$+0.0089$	$+0.0032$
$[32, 64)$	-0.0068	-0.0113
$[64, 128)$	$+0.0000$	-0.0032
$[128, \infty)$	-0.0007	-0.0033

Trên paper-SR12, hard recall proxy tổng thể tăng từ 0.8357 lên 0.8690, precision proxy tăng từ 0.0167 lên 0.0196, trong khi số prediction giảm từ 1 595 279 xuống 1 416 440. Trên default scale set, hard recall proxy tăng từ 0.8243 lên 0.8659, precision proxy tăng từ 0.0153 lên 0.0182, prediction count giảm từ 1 725 098 xuống 1 522 654. Vì vậy ASR+JSAR không chỉ tạo thêm nhiều dự đoán; nó còn làm kết quả trên hard subset gọn hơn ở ngưỡng đánh giá.

1.7 ASR thay đổi phân phối scale như thế nào

Hai scale set có hành vi khác nhau. Default baseline có $E[1/\text{scale}] = 1.270$, extreme zoom-in mass 0.300, extreme zoom-out mass 0.100. Sau ASR, $E[1/\text{scale}]$ giảm còn 1.062, zoom-in mass giảm còn 0.200, zoom-out mass gần như giữ quanh 0.094. Điều này cho thấy ASR không chỉ tăng zoom-in; nó cân bằng lại phân phối để tránh quá nhiều trường hợp làm mặt quá dễ, đồng thời vẫn giữ áp lực lên vùng nhỏ.

Paper-SR12 baseline vốn đã tiny-heavy hơn: $E[1/\text{scale}] = 0.890$, extreme zoom-in mass 0.083, extreme zoom-out mass 0.250. Sau ASR, $E[1/\text{scale}]$ gần như giữ nguyên ở 0.887, extreme zoom-in mass giảm còn 0.058, extreme zoom-out mass giảm còn 0.198. Do đó trên paper-SR12, ASR chủ yếu làm mềm các scale cực đoan thay vì dịch mạnh phân phối. Đây là lý do paper-SR12 baseline đã tốt hơn default baseline trên Hard, nhưng delta bổ sung của ASR+JSAR nhỏ hơn một chút.

Bảng VI.6: Thống kê scale và face-size distribution sau SR simulation.

Thiết lập	$E[1/s]$	Tiny ratio	Median size	Tiny $\rightarrow \geq 16$
Default baseline	1.270	0.475	17.12	0.223
Default ASR+JSAR	1.062	0.523	15.09	0.146
Paper-SR12 baseline	0.890	0.582	12.97	0.092
Paper-SR12 ASR+JSAR	0.887	0.577	13.18	0.081

Bảng trên giúp đọc đúng vai trò của ASR. Với default scale set, ASR dịch face-size distribution về phía tiny nhiều hơn: tiny ratio tăng từ 0.475 lên 0.523, median size giảm từ 17.12 xuống 15.09. Với paper-SR12, baseline đã có tiny ratio rất cao 0.582, nên ASR không đẩy thêm mạnh về tiny mà tinh chỉnh nhẹ để tránh phân phối quá cực đoan. Điều này làm kết quả hợp lý hơn: ASR+JSAR vẫn tăng Hard AP ở cả hai scale set, nhưng mức tăng phụ thuộc headroom của baseline.

1.8 Giới hạn và failure modes

ASR+JSAR có trade-off rõ. Nếu JSAR quá mạnh, fallback positives có thể đưa thêm anchors nhiều và làm precision giảm. Nếu ASR quá mạnh, scale distribution có thể collapse về một vài scale, khiến mô hình overfit tiny faces và mất cân bằng Easy/Medium. Trong thiết lập hiện tại, probability floor 0.03, EMA 0.8, warm-up 2

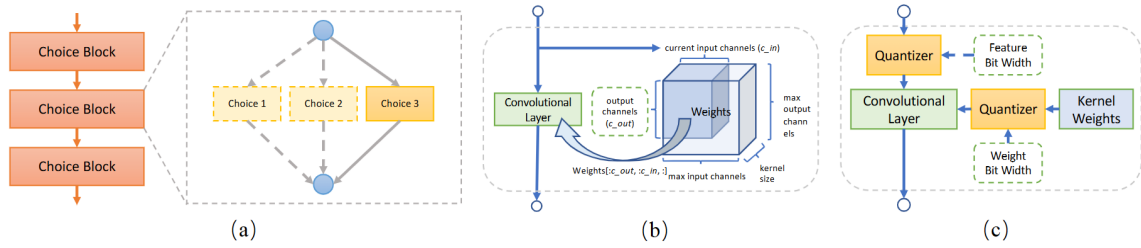
epoch và fallback top-k 4 là các cơ chế để giảm rủi ro đó. Kết quả thực nghiệm cho thấy trade-off vẫn tồn tại: Easy/Medium giảm nhẹ, nhưng Hard tăng đủ lớn để mAP tăng.

Giới hạn thứ hai là ASR+JSAR hiện được kiểm chứng trên WIDER FACE với SCRFD-2.5GF. Nó chứng minh cải tiến có ý nghĩa trong miền tiny/hard faces của WIDER FACE, nhưng chưa chứng minh tính tổng quát trên dataset khác hoặc trên các backbone/FLOPs regime khác. Nếu mở rộng tiếp, hướng hợp lý là ablation riêng ASR-only, JSAR-only, hybrid-fallback vs soft-weight, và thử trên các mức SCRFD nhỏ hơn như 0.5GF để xem tiny supervision có còn giúp khi ngân sách cực thấp hay không.

2 SPOS-NAS như cải tiến phân tích kiến trúc

Song song với official reproduction, nhóm thực hiện thêm một cải tiến theo hướng SPOS-NAS để phân tích trực tiếp nguyên lý *Computation Redistribution* trong điều kiện tài nguyên giới hạn. Phần này không thay thế kết quả AP thật của SCRFD, nhưng là đóng góp thực nghiệm riêng của nhóm: thay vì chỉ lặp lại mô hình có sẵn, nhóm xây dựng một supernet weight-sharing, thiết kế proxy heatmap task cho face detection, rồi dùng nó để khảo sát hàng nghìn kiến trúc dưới ràng buộc 2.5 GFLOPs. Mục tiêu của phần này là trả lời câu hỏi: nếu search được đặt dưới áp lực nhận diện khuôn mặt nhỏ, nó có tự học cách phân bổ compute giống tinh thần SCRFD hay không.

Quá trình này được minh họa cụ thể qua Hình VI.1.



Hình VI.1: Minh họa luồng làm việc của SPOS (hình gốc từ bài báo SPOS): (a) Tại mỗi stage có nhiều "khối lựa chọn", trong mỗi lượt truyền (forward pass), supernet sẽ chọn ngẫu nhiên một khối để kích hoạt; (b) Cơ chế cắt trọng số (weight slicing) được kích hoạt khi hai khối ở hai tầng liên tiếp có kích thước in/out khác nhau, khối lớn hơn sẽ tự động cắt bớt số kênh cho tương thích; (c) Kỹ thuật mixed precision được tích hợp trong quá trình huấn luyện nhằm tối ưu bộ nhớ và thời gian.

Thí nghiệm proxy dùng WIDER FACE với 12 876 ảnh train và 3 222 ảnh validation. Không gian tìm kiếm xét bốn stage C_2, C_3, C_4, C_5 , mỗi stage có 15 lựa chọn số kênh từ 8 đến 120 và 4 lựa chọn độ sâu, tức $60^4 = 12\,960\,000$ cấu hình lý thuyết. Sau khi lọc theo ràng buộc ≤ 2.5 GFLOPs, còn 2670925 cấu hình hợp lệ. Vì không thể train đầy đủ từng kiến trúc, nhóm huấn luyện một supernet SPOS bằng proxy heatmap task rồi đánh giá 1 000 kiến trúc được lấy mẫu từ không gian hợp lệ.

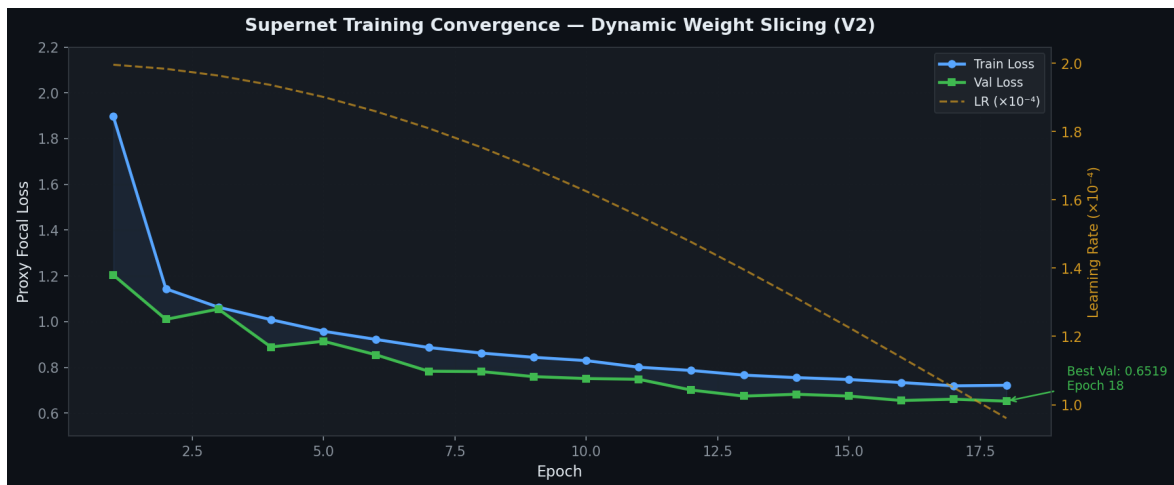
Bảng VI.7: Thiết lập chính của thí nghiệm SPOS-NAS proxy.

Mục	Giá trị
Phần cứng	Kaggle T4 GPU 16GB, giới hạn khoảng 6 giờ
Dataset	WIDER FACE, 12 876 ảnh train và 3 222 ảnh validation
Mục tiêu proxy	Heatmap-based surrogate objective cho phát hiện khuôn mặt
Epoch supernet	35 epoch
Optimizer	AdamW, learning rate 2×10^{-4} , weight decay 10^{-4}
Schedule	Cosine annealing với $\eta_{\min} = 10^{-6}$
Batch size hiệu dụng	64 nhờ mixed precision
Loss	Proxy focal loss, $\alpha = 0.25, \gamma = 2.0$
Ràng buộc kiến trúc	Tối đa 2.5 GFLOPs
Số kiến trúc đánh giá	1 000 kiến trúc, khoảng 5–8 giây mỗi kiến trúc

Ban đầu, để đơn giản quá trình cài đặt thì nhóm chúng em dùng cơ chế zeropadding và làm nhiều block ở một tầng. Nên thay đổi quan trọng trong quá trình phát triển proxy là thay cơ chế zero-padding tĩnh bằng dynamic weight slicing (giống với cách làm ban đầu của tác giả). Cách cũ làm gradient bị pha loãng vì các cấu hình nhỏ phải đi qua tensor có nhiều kênh đệm bằng 0; cách mới chỉ lát cắt đúng phần trọng số đang hoạt động, nhờ đó gradient tập trung vào sub-network được lấy mẫu. Kết quả trong file thí nghiệm cho thấy ngưỡng validation loss < 0.70 đạt vào khoảng epoch 12, trong khi phiên bản cũ cần khoảng epoch 17–18; tức nhanh hơn xấp xỉ 30% theo số epoch tới ngưỡng hội tụ.

Bảng VI.8: Tóm tắt kết quả huấn luyện supernet proxy.

Chỉ số	Kết quả
Best validation loss	0.6519 tại epoch 18
Thời gian epoch sau warm-up	Khoảng 247 giây/epoch
Mốc hội tụ < 0.70	Khoảng epoch 12 với dynamic weight slicing
Cải thiện so với zero-padding	Giảm khoảng 30% số epoch tới ngưỡng 0.70



Hình VI.2: Đường cong hội tụ của supernet proxy; best validation loss đạt 0.6519 tại epoch 18.

Đường cong train/validation cho thấy giai đoạn đầu học rất nhanh: train loss giảm từ 1.8963 ở epoch 1 xuống 1.1431 ở epoch 2, còn validation loss giảm từ 1.2034 xuống 1.0095. Sau đó loss tiếp tục giảm nhưng có dao động nhỏ, đúng với bản chất SPOS vì mỗi mini-batch có thể kích hoạt một sub-network khác nhau. Các mốc đáng chú ý là validation loss 0.7004 tại epoch 12, 0.6739 tại epoch 13, 0.6548 tại epoch 16 và tốt nhất 0.6519 tại epoch 18. Vì vậy, nhóm không chỉ quan sát một điểm cuối tốt mà còn thấy quá trình hội tụ có hình dạng hợp lý.

Sau khi huấn luyện supernet, nhóm đánh giá 1000 kiến trúc hợp lệ bằng proxy loss. Kiến trúc tốt nhất là Arch #980 với validation loss 0.6521, trong khi phân bố toàn bộ 1000 kiến trúc có mean 0.7519, standard deviation 0.0422, worst loss 0.9013, ngưỡng Top-10% là 0.7070, và ngưỡng Top-1% là 0.6656. Điều này cho thấy search không chỉ tìm được một điểm may mắn đơn lẻ mà còn tạo ra một phân tầng chất lượng khá rõ giữa nhóm tốt và nhóm kém.

Bảng VI.9: Top-10 kiến trúc tốt nhất trong thí nghiệm SPOS-NAS.

Rank	C_2	C_3	C_4	C_5	Val Loss
1 / #980	(48, 3)	(32, 1)	(72, 4)	(80, 4)	0.6521
2 / #116	(48, 3)	(64, 3)	(104, 1)	(56, 4)	0.6558
3 / #837	(48, 3)	(80, 1)	(96, 2)	(88, 4)	0.6585
4 / #766	(48, 3)	(32, 1)	(16, 3)	(56, 4)	0.6619
5 / #514	(48, 3)	(56, 4)	(72, 1)	(80, 3)	0.6627
6 / #475	(64, 2)	(64, 3)	(24, 1)	(48, 4)	0.6630
7 / #331	(56, 3)	(32, 4)	(88, 1)	(96, 3)	0.6656
8 / #204	(48, 3)	(56, 4)	(64, 1)	(48, 2)	0.6695
9 / #879	(48, 3)	(56, 1)	(88, 4)	(40, 4)	0.6713
10 / #732	(48, 3)	(40, 4)	(96, 4)	(96, 3)	0.6724

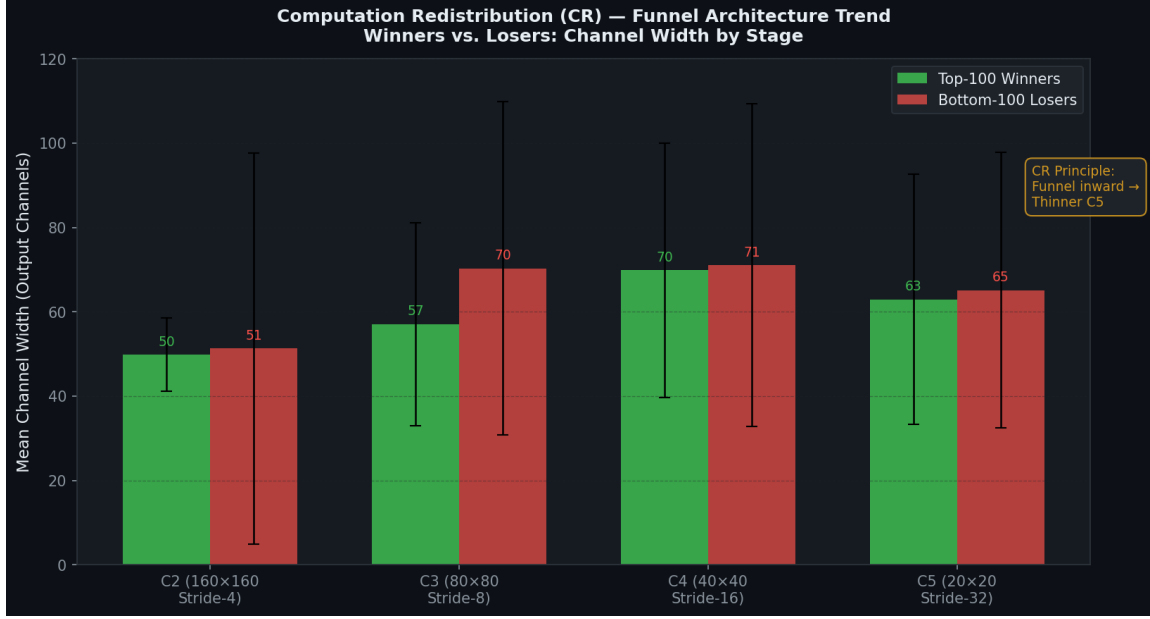


Hình VI.3: Phân bố validation loss của 1000 kiến trúc NAS trong thí nghiệm proxy tài nguyên giới hạn.

Quan sát quan trọng nhất từ leaderboard là 8/10 kiến trúc đứng đầu cùng chọn $C_2 = (48, d = 3)$. Nói cách khác, dưới áp lực của Sample Redistribution hướng vào mặt nhỏ, search tự tạo ra một đồng thuận mạnh ở stage nông nhất. Điều này phù hợp với trực giác SCRFD: khuôn mặt nhỏ cần đặc trưng độ phân giải cao, nên C_2 không được quá mỏng dù chi phí tính toán ở stage này đắt.

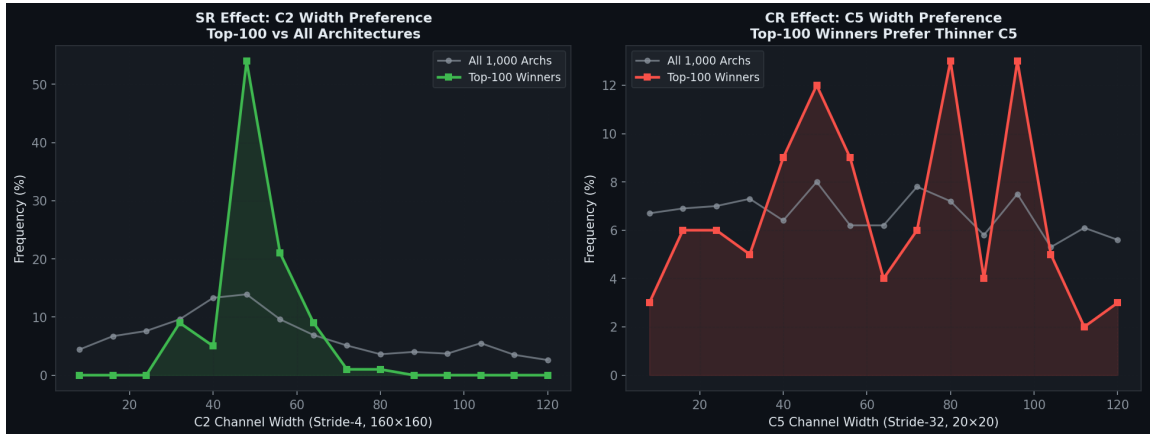
Bảng VI.10: Phân bố chi phí tính toán giữa Top-100 winners và nhóm thua cùng mức GFLOPs.

Stage	Spatial res.	Top-100	Nhóm thua cùng GFLOPs	Delta
C_2	160×160	1.124	0.643	+0.481
C_3	80×80	0.655	1.102	-0.447
C_4	40×40	0.226	0.277	-0.051
C_5	20×20	0.078	0.060	+0.018



Hình VI.4: Phân tích Funnel Principle: Top-100 winners đầu tư nhiều hơn vào C_2 và tránh bẫy tiêu tốn quá nhiều FLOPs ở C_3 .

Bảng phân bố GFLOPs làm rõ hơn ý nghĩa của Computation Redistribution. Nhóm thắng không đơn giản là “dùng nhiều compute ở stage nông hơn” một cách mù quáng; chúng tăng mạnh C_2 nhưng kiểm soát C_3 , vì C_3 vẫn còn ở độ phân giải 80×80 nên mọi tăng width/depth đều rất đắt. Nhóm thua rơi vào một dạng C_3 trap: đổ 1.102 GFLOPs vào C_3 , khiến C_2 chỉ còn 0.643 GFLOPs và làm yếu tầng nền tảng cho mặt nhỏ. Ngược lại, Top-100 winners dành 1.124 GFLOPs cho C_2 , giữ C_3 vừa đủ, và vẫn còn một phần nhỏ cho C_5 để duy trì ngữ nghĩa toàn cục.



Hình VI.5: Phân bố độ rộng C_2 và C_5 : nhóm Top-100 tập trung quanh $C_2 \in \{40, 48, 56\}$ và tránh C_5 quá rộng.

2.1 BatchNorm recalibration và độ tin cậy của xếp hạng

Một chi tiết quan trọng để xếp hạng NAS đáng tin hơn là BatchNorm recalibration. Trong SPOS, running mean và running variance của BatchNorm được tích lũy trên phân phối hỗn hợp của rất nhiều sub-network. Nếu lấy trực tiếp các thống kê trung bình đó để đánh giá một kiến trúc cụ thể, dự đoán có thể bị lệch vì kiến trúc đang test có width/depth khác với “kiến trúc trung bình” mà BatchNorm đã thấy trong quá trình train. Vì vậy, trước khi chấm proxy loss cho từng candidate, cần chạy vài batch calibration để cập nhật lại thống kê BatchNorm cho riêng candidate đó, rồi mới chuyển sang chế độ đánh giá.

Nếu bỏ bước này, lỗi xếp hạng có thể tăng mạnh với các kiến trúc có dung lượng lệch nhiều so với trung bình của supernet. Đây là lý do trong protocol search, mỗi candidate không chỉ được forward một lần để lấy loss, mà còn cần một bước đồng bộ thống kê ngắn. Điểm này làm phần cải tiến đáng tin hơn: kết quả Top-100 không chỉ phản ánh noise của weight sharing, mà đã giảm bớt sai lệch do BatchNorm dùng thống kê không phù hợp.

2.2 Phạm vi search và giới hạn của proxy

Mặc dù 1 000 kiến trúc là đủ để thấy xu hướng lớn, nó chỉ chiếm khoảng 0.037% trong 2 670 925 cấu hình hợp lệ. Vì vậy, kết quả hiện tại mạnh ở vai trò phát hiện quy luật: C_2 có một vùng tối ưu rõ quanh 40–56 kênh, Top-100 tránh C_5 quá rộng, và các kiến trúc thua thường rơi vào bất tiêu tốn quá nhiều compute ở C_3 . Tuy nhiên, con số này chưa đủ để khẳng định đã tìm được global optimum trong toàn bộ không gian 2.5 GFLOPs. Một search 10 000 kiến trúc hoặc search đa mục tiêu theo cả proxy loss, GFLOPs và latency sẽ cho độ tin cậy thống kê cao hơn.

Proxy heatmap loss cũng không phải AP thật. Nó là một mục tiêu thay thế được thiết kế để phản ánh khả năng giữ tín hiệu mặt nhỏ trong feature cascade. Do đó, kết luận đúng mức là: proxy xác nhận định tính nguyên lý Computation Redistribution, còn bước cần làm tiếp là huấn luyện end-to-end các kiến trúc top với full detection head gồm classification, bbox regression và IoU-related loss để đo AP Easy/Medium/Hard trên WIDER FACE.

2.3 Edge-Focus: kiểm chứng search phụ thuộc vào mục tiêu

Một thí nghiệm bổ sung trong file kết quả thay đổi proxy target từ điểm tâm khuôn mặt sang biên hộp mặt. Khi mục tiêu chỉ là điểm tâm, mô hình cần đặc trưng cục bộ mạnh ở độ phân giải cao, nên C_2 được ưu tiên. Khi mục tiêu là đường biên, mô hình cần receptive field lớn hơn để nối các cạnh thành cấu trúc hình học nhất quán, nên search có xu hướng chuyển compute sang các stage sâu hơn. Đây là kiểm chứng quan trọng: SPOS-NAS không chỉ học thuộc một template SCRFD cố định, mà phản ứng theo bản chất hình học của mục tiêu huấn luyện.

Bảng VI.11: Dịch chuyển chi phí tính toán khi đổi proxy target từ center-focus sang edge-focus.

Stage	Center-focus winners	Edge-focus winners	Nhận xét
C_2	1.124	0.729	Giảm khoảng 35%, bớt phụ thuộc vào chi tiết cục bộ
C_3	0.655	0.886	Tăng khoảng 35%, cần receptive field lớn hơn
C_4	0.226	0.392	Tăng khoảng 73%, hỗ trợ cấu trúc biên dài hơn
C_5	0.078	0.114	Tăng khoảng 46%, bổ sung ngữ nghĩa toàn cục

Kết quả này làm nhận xét về C_3 trap sắc hơn. Trong center-focus, đầu tư quá nhiều vào C_3 là lãng phí vì mục tiêu chỉ cần phát hiện vùng cục bộ của mặt nhỏ; khi đó C_3 đắt mà không trực tiếp giúp tín hiệu tiny-face bằng C_2 . Nhưng với edge-focus, C_3 không còn là bất theo nghĩa tuyệt đối: nó trở thành nơi cần thiết để gom thông tin hình học rộng hơn. Như vậy, kết luận đúng không phải là “luôn tăng C_2 , luôn giảm C_3 ”, mà là kiến trúc tốt phải phân bổ compute theo yêu cầu hình học của task. Đây chính là cách diễn giải sâu hơn của Computation Redistribution.

2.4 Kết luận về cải tiến SPOS-NAS

Từ góc nhìn của report cuối kỳ, giá trị của SPOS-NAS nằm ở ba điểm. Thứ nhất, nó chứng minh có thể khảo sát không gian 2.5 GFLOPs rất lớn trong ngân sách hạn chế bằng weight sharing, thay vì chỉ thảo luận NAS trên lý thuyết. Thứ hai, các kết quả định lượng ủng hộ trực giác SCRFD: nhóm tốt đầu tư mạnh vào C_2 , tránh C_5 quá rộng và kiểm soát chi phí C_3 . Thứ ba, thí nghiệm edge-focus cho thấy kết quả search thay đổi theo mục tiêu proxy, nên NAS đang phản ứng với nhu cầu biểu diễn của task chứ không đơn giản ghi nhớ một khuôn mẫu.

Giới hạn còn lại là proxy heatmap loss vẫn chỉ là surrogate, nên kết quả này không được dùng thay cho AP thật trên WIDER FACE. Hướng tiếp theo hợp lý là train end-to-end Arch #980 bằng full SCRFD detection head trong khoảng 300 epoch, thử BN-free supernet bằng Group Normalization để giảm phụ thuộc vào recalibration, mở rộng search lên 10 000 kiến trúc, và làm multi-objective NAS theo proxy loss, GFLOPs, latency để tìm kiến trúc phù hợp phần cứng cụ thể.

Chương VII

KẾT LUẬN

Qua việc đọc paper, đối chiếu code và chạy lại thực nghiệm, insight trung tâm của SCRFD có thể tóm lại như sau: với face detection hiệu quả, tổng FLOPs không quan trọng bằng vị trí FLOPs và chất lượng tín hiệu huấn luyện mà từng stage nhận được. Một detector 2.5GF không tự động tốt vì nó có 2.5GF; nó tốt khi compute được đặt đúng vào các mức feature còn giữ chi tiết không gian cho mặt nhỏ, và khi pipeline huấn luyện tạo đủ supervision cho những khuôn mặt tiny/hard vốn quyết định phần khó nhất của WIDER FACE.

Kết quả tái hiện củng cố nhận định đó. Baseline paper-SR12 cosine 80e đạt Easy 91.61, Medium 89.99, Hard 73.71, thấp hơn số official *SCRFD-2.5GF* lần lượt 2.17, 2.17 và 4.16 điểm. Gap lớn nhất nằm ở Hard, tức đúng vùng mà SCRFD nhắm tới. Điều này cho thấy phần khó khi tái hiện SCRFD không chỉ là chạy được model, mà là tái tạo đủ chính xác toàn bộ chuỗi thiết kế: kiến trúc searched, scale redistribution, assignment, lịch huấn luyện và checkpoint selection. Dù chưa khớp hoàn toàn model zoo, reproduction vẫn đủ giá trị vì nó tạo baseline kiểm soát để phân tích các thay đổi tiếp theo.

Hai phần mở rộng đều đi về cùng một kết luận nhưng từ hai phía khác nhau. ASR+JSAR chứng minh rằng giữ nguyên kiến trúc suy luận vẫn có thể cải thiện Hard AP nếu training signal cho tiny faces được sửa đúng chỗ: Hard AP trên paper-SR12 tăng từ 73.71 lên 77.38, trong khi tiny positives/GT tăng khoảng $1.52\times$. SPOS-NAS lại nhìn từ phía kiến trúc: khi search dưới ràng buộc 2.5GF, các cấu hình tốt có xu hướng đầu tư mạnh vào C_2 , kiểm soát C_3 , tránh C_5 quá rộng và phản ứng khác đi khi đổi proxy target sang edge-focus. Vì vậy, Computation Redistribution không nên hiểu là một công thức cố định kiểu “tăng stage nông”, mà là nguyên tắc phân bổ compute theo yêu cầu hình học của task.

Giới hạn quan trọng nhất là reproduction chưa tái tạo toàn bộ pipeline model zoo và phần SPOS-NAS vẫn dùng proxy heatmap loss thay vì AP end-to-end. Tuy nhiên, các kết quả hiện có đã tạo thành một bức tranh nhất quán: SCRFD mạnh vì nó tối ưu đồng thời nơi mô hình đặt năng lực biểu diễn và nơi dữ liệu tạo tín hiệu học. Hướng tiếp theo hợp lý là train end-to-end các kiến trúc NAS tốt nhất bằng full SCRFD head, làm ablation riêng ASR-only/JSAR-only, và kiểm tra các cơ chế này trên dataset hoặc mức FLOPs khác để đánh giá tính tổng quát ngoài WIDER FACE.

Tài liệu tham khảo

- [1] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, “Sample and computation redistribution for efficient face detection,” in *International Conference on Learning Representations (ICLR)*, 2022. Accessed: Feb. 25, 2026. [Online]. Available: <https://openreview.net/forum?id=RhB1AdoFfGE>
- [2] DeepInsight contributors, *Insightface scrfd repository and documentation*, <https://github.com/deepinsight/insightface/tree/master/detection/scrfd>, Accessed 2026-04-19, 2026.
- [3] S. Yang, P. Luo, C. C. Loy, and X. Tang, “Wider face: A face detection benchmark,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5525–5533. DOI: 10.1109/CVPR.2016.596
- [4] Z. Guo et al., “Single path one-shot neural architecture search with uniform sampling,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 544–560. DOI: 10.1007/978-3-030-58598-3_32
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91
- [6] W. Liu et al., “Ssd: Single shot multibox detector,” *arXiv preprint arXiv:1512.02325*, 2016. Accessed: Feb. 26, 2026. [Online]. Available: <https://arxiv.org/abs/1512.02325>
- [7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal loss for dense object detection,” in *2017 IEEE International Conference on Computer Vision*, 2017, pp. 2999–3007. DOI: 10.1109/ICCV.2017.324
- [8] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, “Retinaface: Single-shot multi-level face localisation in the wild,” *arXiv preprint arXiv:1905.00641*, 2020. Accessed: Feb. 26, 2026. [Online]. Available: <https://arxiv.org/abs/1905.00641>